LAPORAN PRAKTIKUM

POSTTEST 3

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Pirlo Syabila Hafuza (2509106008)

Kelas (A1 ‘25)

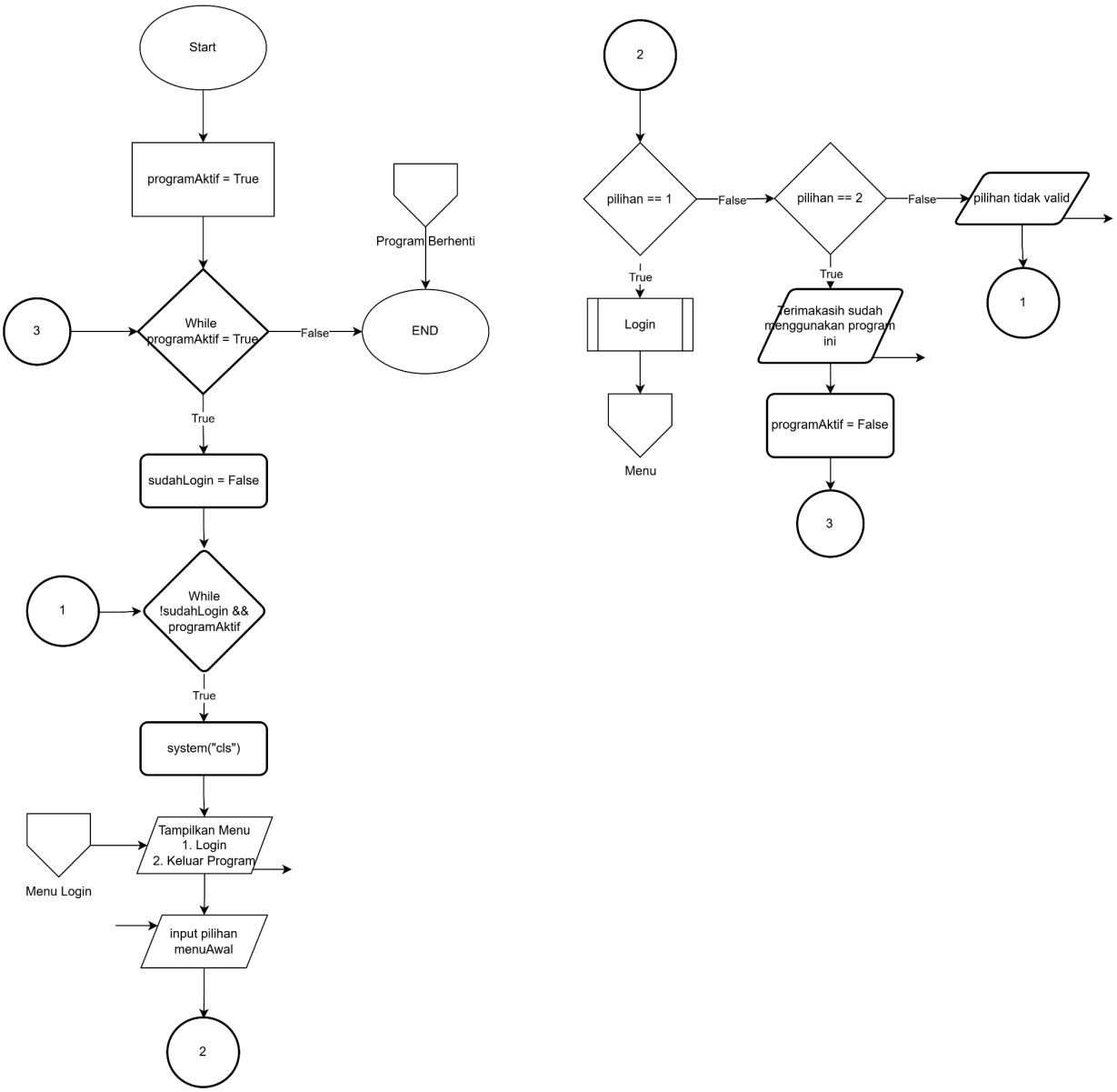
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

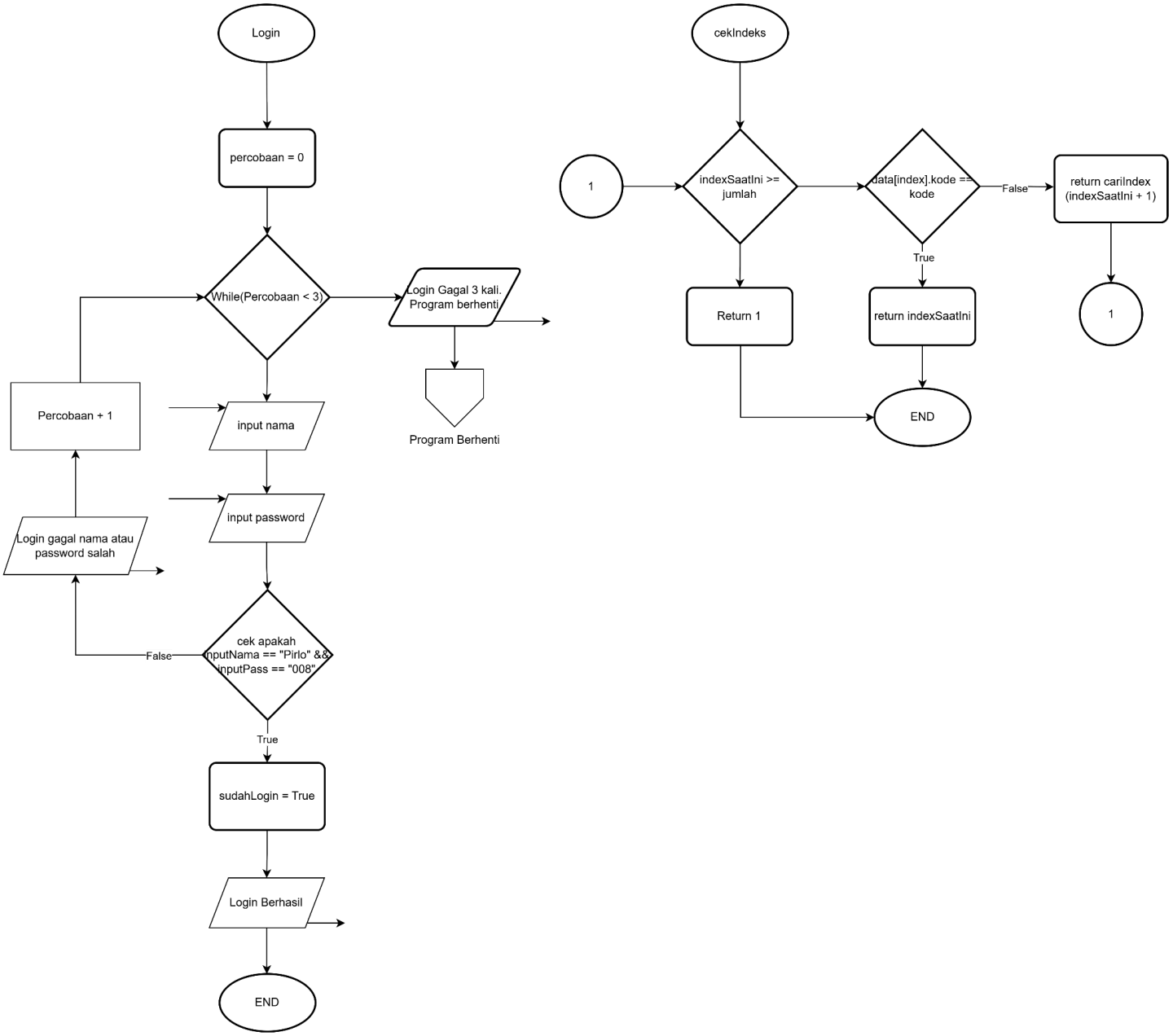
SAMARINDA

2026

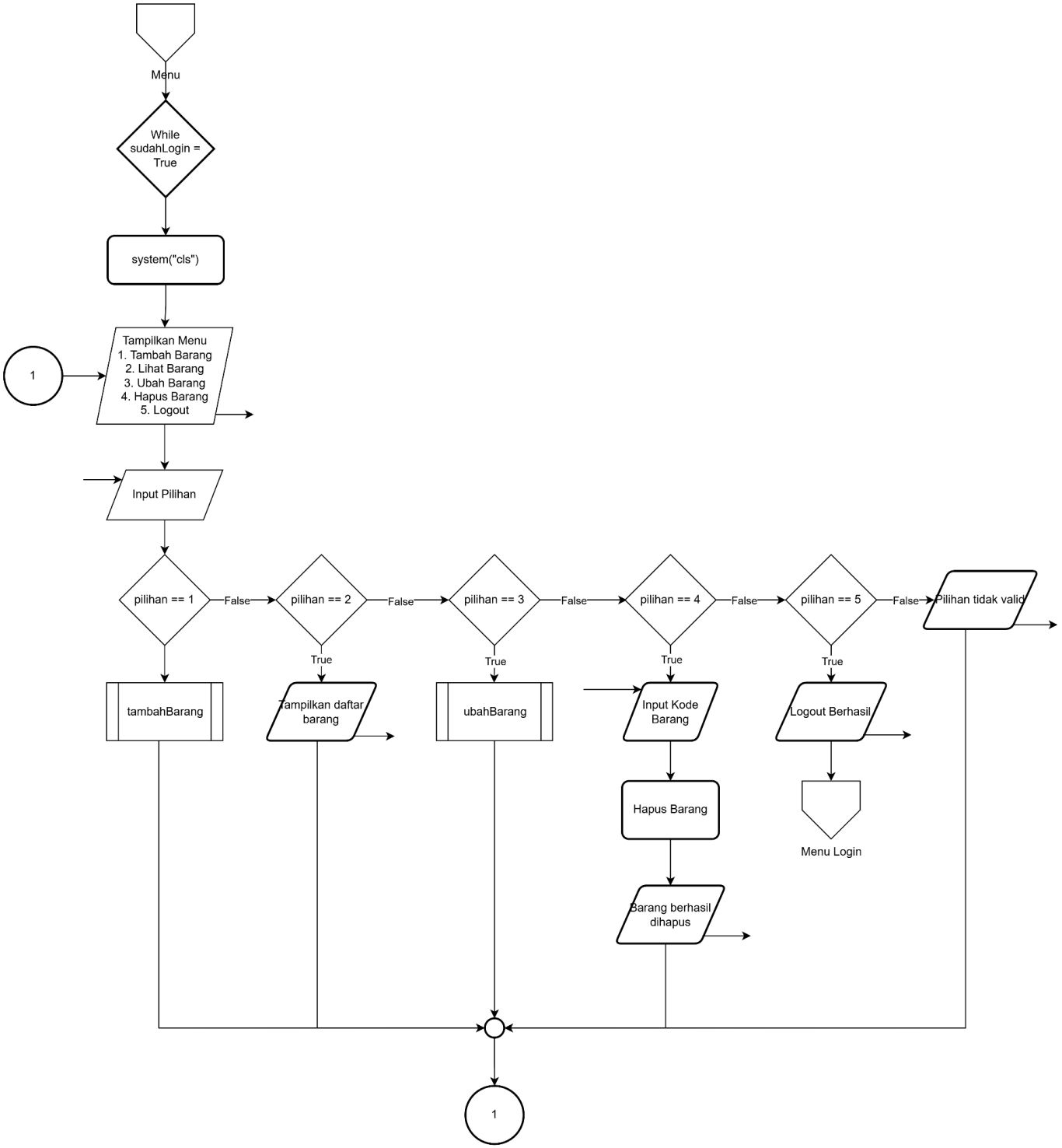
1. Flowchart



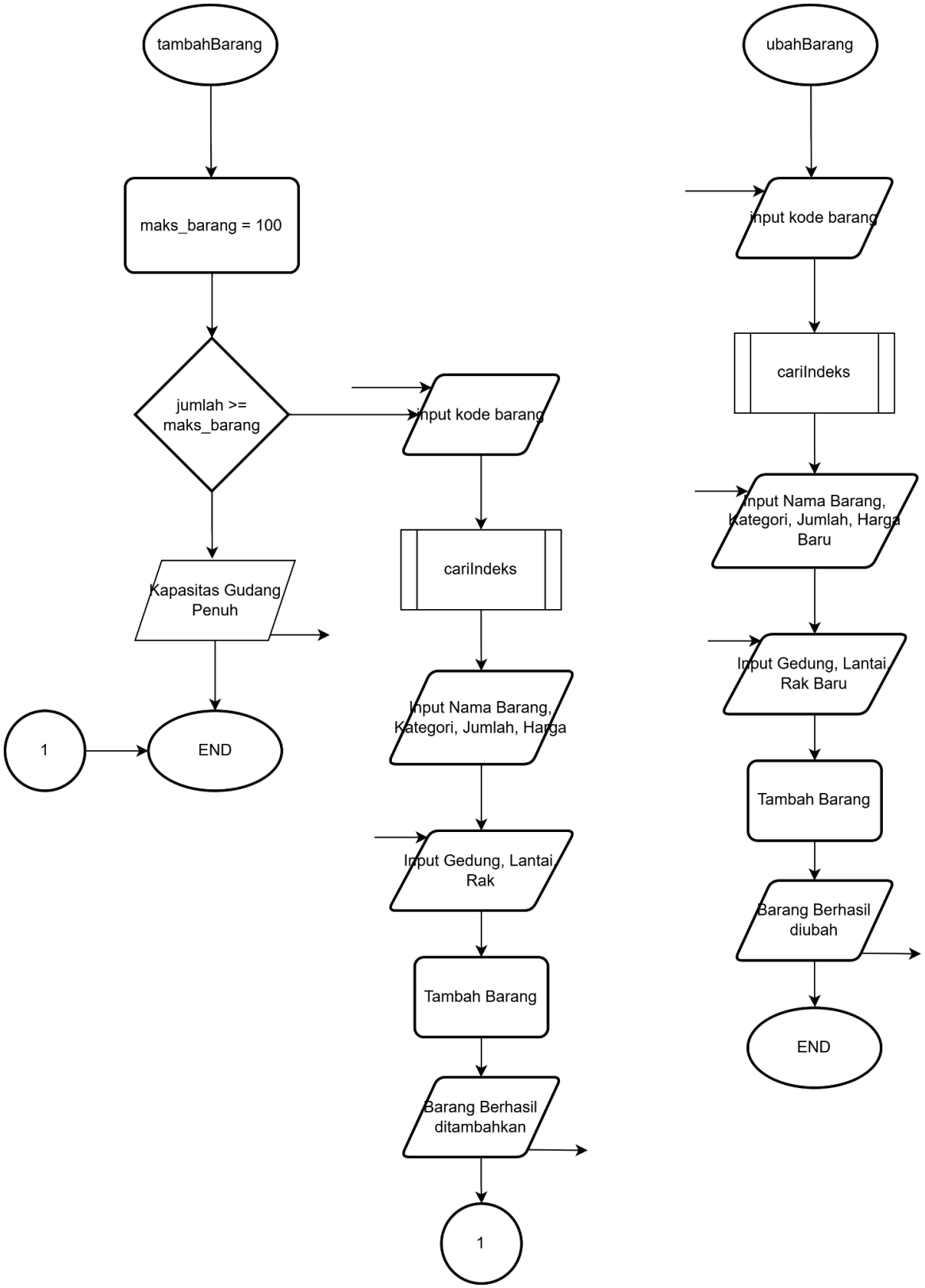
Gambar 1.1 Flowchart



Gambar 1.2 Flowchart



Gambar 1.3 Flowchart



Gambar 1.4 Flowchart

2. Deskripsi Singkat Program

Program ini merupakan aplikasi manajemen inventaris gudang elektronik yang dilengkapi dengan sistem login. Pengguna harus memasukkan username dan password dengan batas maksimal tiga kali percobaan sebelum akses diberikan. Setelah berhasil login, tersedia menu utama yang akan terus berjalan hingga pengguna memilih opsi logout. Program ini menyediakan fitur tambah, lihat, ubah, dan hapus data barang lengkap dengan informasi lokasi penyimpanan.

3. Source Code

A. Fitur Login

Fitur Login menggunakan fungsi rekursif `prosesLogin` dengan parameter `percobaan`. Jika `percobaan >= 3`, fungsi mengembalikan `false` dan program berhenti. Jika belum, program meminta input username dan password; jika sesuai ("Pirlo" dan "008"), login berhasil dan fungsi mengembalikan `true`. Jika salah, fungsi memanggil dirinya sendiri dengan `percobaan + 1` untuk mengulangi proses hingga berhasil atau percobaan habis.

Source Code:

```
bool prosesLogin(int percobaan) { // Fungsi Rekursif
    if (percobaan >= 3) {
        cout << "\nGagal login 3 kali. Program berhenti.";
        return false;
    }

    string inputNama, inputPass;
    cout << "\n--- Login (Percobaan " << percobaan + 1 << "/3) ---";
    cout << "\nUsername: ";
    cin >> inputNama;
    cout << "Password: ";
    cin >> inputPass;

    if (inputNama == "Pirlo" && inputPass == "008") {
        cout << "\nLogin Berhasil! Selamat datang." << endl;
        cout << "Tekan Enter untuk melanjutkan...";
        cin.ignore();
        cin.get();
        return true;
    } else {
```

```
        cout << "Username atau Password salah!" << endl;
        cout << "Tekan Enter untuk mencoba lagi...";
        cin.ignore();
        cin.get();
        return prosesLogin(percobaan + 1);
    }
}
```

B. Fitur Tampilan Menu

Prosedur `tampilkanMenuUtama` hanya berfungsi menampilkan daftar pilihan menu (1-5). Fungsi `main` bertugas sebagai pengendali utama program: menginisialisasi data, mengatur alur login, dan menampilkan menu utama dalam sebuah loop. Berdasarkan input pengguna, `main` hanya memanggil fungsi-fungsi lain seperti `tambahBarang`, `lihatBarang`, `ubahBarang`, `hapusBarang`, atau menangani proses logout dan keluar program.

Source Code:

```
void tampilkanMenuUtama() { // Prosedur
    cout << "\n1. Tambah Barang";
    cout << "\n2. Lihat Barang";
    cout << "\n3. Ubah Barang";
    cout << "\n4. Hapus Barang";
    cout << "\n5. Logout";
    cout << "\n\nPilihan: ";
}

int main() { // Fungsi Utama
    const int MAKS_BARANG = 100;
    ItemInventaris dataBarang[MAKS_BARANG];
    int jumlahBarang = 0;

    dataBarang[0] = {"EL001", "Laptop Asus", "Komputer", 10, 5000000, {"Gudang
A", "1", "01"}, true};
    dataBarang[1] = {"EL002", "Mouse Logitech", "Aksesoris", 50, 150000,
{"Gudang A", "1", "02"}, true};
    dataBarang[2] = {"EL003", "Monitor LG", "Komputer", 0, 2000000, {"Gudang B",
"2", "05"}, false};
    jumlahBarang = 3;

    bool programAktif = true;

    while (programAktif) {
        bool sudahLogin = false;
        int menuAwal;
```

```
    while (!sudahLogin && programAktif) {
        system("cls");
        cout << "\n==================================================";
        cout << "\n   SISTEM MANAJEMEN INVENTARIS GUDANG ELEKTRONIK";
        cout << "\n==================================================";
        cout << "\n1. Login";
        cout << "\n2. Keluar Program";
        cout << "\n\nPilihan: ";
        cin >> menuAwal;
        cin.ignore();

        if (menuAwal == 1) {
            sudahLogin = prosesLogin(0);
        } else if (menuAwal == 2) {
            cout << "\nTerima kasih telah menggunakan program ini.";
            programAktif = false;
        } else {
            cout << "\nPilihan tidak valid!";
            cout << "\nTekan Enter untuk melanjutkan...";
            cin.get();
        }
    }

    while (sudahLogin) {
        system("cls");
        cout << "\n==================================================";
        cout << "\n   MENU UTAMA INVENTARIS GUDANG ELEKTRONIK";
        cout << "\n==================================================";
        tampilkanMenuUtama();

        int menuUtama;
        cin >> menuUtama;
        cin.ignore();

        if (menuUtama == 1) {
            tambahBarang(dataBarang, jumlahBarang, MAKS_BARANG);
        } else if (menuUtama == 2) {
            lihatBarang(dataBarang, jumlahBarang);
        } else if (menuUtama == 3) {
            ubahBarang(dataBarang, jumlahBarang);
        } else if (menuUtama == 4) {
            hapusBarang(dataBarang, jumlahBarang);
        } else if (menuUtama == 5) {
            cout << "\nLogout berhasil.";
            cout << "\nTekan Enter untuk keluar...";
            cin.get();
            sudahLogin = false;
        } else {
            cout << "\nPilihan tidak valid.";
            cout << "\nTekan Enter untuk melanjutkan...";
            cin.get();
```

```
            }
        }
    }

    return 0;
}
```

C. Fitur Tambah Barang

Prosedur `tambahBarang` berfungsi menambahkan data inventaris baru ke dalam array. Pertama, program mengecek apakah gudang sudah penuh (`jumlah >= maks`); jika penuh, proses dibatalkan. Jika masih ada ruang, program meminta input `kodeBarang` dan memvalidasinya menggunakan fungsi `cariIndex` agar tidak ada kode duplikat. Selanjutnya, program menginput detail barang (nama, kategori, jumlah, harga) serta lokasi penyimpanan (gedung, lantai, rak), lalu mengatur status `tersedia` berdasarkan nilai jumlah. Terakhir, variabel `jumlah` ditambah 1 dan pesan sukses ditampilkan.

Source Code:

```
void tambahBarang(ItemInventaris data[], int &jumlah, int maks) { // Prosedur
    cout << "\n==================================================";
    cout << "\n   TAMBAH BARANG";
    cout << "\n==================================================";

    if (jumlah >= maks) {
        cout << "\nKapasitas gudang penuh!";
        cout << "\nTekan Enter untuk melanjutkan...";
        cin.get();
        return;
    }

    cout << "\nKode Barang: ";
    cin >> data[jumlah].kodeBarang;

    if (cariIndex(data, 0, jumlah, data[jumlah].kodeBarang) != -1) {
        cout << "Kode barang sudah digunakan!";
        cout << "\nTekan Enter untuk melanjutkan...";
        cin.ignore();
        cin.get();
        return;
    }

    cin.ignore();
    cout << "Nama Barang: ";
    getline(cin, data[jumlah].namaBarang);
```

```
    cout << "Kategori: ";
    getline(cin, data[jumlah].kategori);
    cout << "Jumlah: ";
    cin >> data[jumlah].jumlah;
    cout << "Harga: ";
    cin >> data[jumlah].harga;
    cin.ignore();

    cout << "\n--- Lokasi Penyimpanan ---";
    cout << "\nGedung: ";
    getline(cin, data[jumlah].alamat.gedung);
    cout << "Lantai: ";
    getline(cin, data[jumlah].alamat.lantai);
    cout << "Rak: ";
    getline(cin, data[jumlah].alamat.rak);

    data[jumlah].tersedia = (data[jumlah].jumlah > 0);
    jumlah++;

    cout << "\nBarang berhasil ditambahkan!";
    cout << "\nTekan Enter untuk melanjutkan...";
    cin.get();
}
```

D. Fitur Lihat Barang

Prosedur `lihatBarang` menampilkan daftar inventaris gudang dalam format tabel. Jika tidak ada data (`jumlah == 0`), program menampilkan pesan "Belum ada data barang". Jika ada data, program membuat tabel, kemudian mengisi setiap baris dengan informasi barang (kode, nama, kategori, jumlah, harga, lokasi gedung-lantai-rak, serta status "Tersedia" atau "Habis" berdasarkan nilai `tersedia`).

Source Code:

```
void lihatBarang(ItemInventaris data[], int jumlah) { // Prosedur
    cout << "\n==================================================";
    cout << "\n   DAFTAR INVENTARIS GUDANG";
    cout << "\n==================================================";
    cout << "\n";

    if (jumlah == 0) {
        cout << "\nBelum ada data barang.";
    } else {
        Table table;
```

```
        table.add_row({"Kode", "Nama Barang", "Kategori", "Jumlah", "Harga",
"Lokasi", "Status"});

        for (size_t i = 0; i < table[0].size(); ++i) {
            table[0][i].format()
                .font_align(FontAlign::center)
                .font_style({FontStyle::bold});
        }

        for (int i = 0; i < jumlah; i++) {
            string lokasi = data[i].alamat.gedung + "-" +
                            data[i].alamat.lantai + "-" +
                            data[i].alamat.rak;
            string status = data[i].tersedia ? "Tersedia" : "Habis";

            table.add_row({
                data[i].kodeBarang,
                data[i].namaBarang,
                data[i].kategori,
                to_string(data[i].jumlah),
                "Rp " + to_string(data[i].harga),
                lokasi,
                status
            });
        }

        cout << table << endl;
    }

    cout << "\nTekan Enter untuk melanjutkan...";
    cin.get();
}
```

E. Fitur Ubah Data Barang

Prosedur `ubahBarang` berfungsi memperbarui data barang yang sudah ada. Program pertama-tama mengecek apakah data kosong (`jumlah == 0`), lalu meminta input `kodeCari` untuk mencari barang menggunakan fungsi `cariIndex`. Jika barang ditemukan (`posisiBarang != -1`), program mengizinkan pengguna menginput ulang seluruh detail barang (nama, kategori, jumlah, harga, dan lokasi penyimpanan), kemudian memperbarui status `tersedia` berdasarkan nilai jumlah. Jika barang tidak ditemukan, program menampilkan pesan kesalahan. Terakhir, program menampilkan konfirmasi hasil operasi dan menunggu pengguna menekan Enter untuk melanjutkan.

Source Code:

```
void ubahBarang(ItemInventaris data[], int jumlah) { // Prosedur
    cout << "\n==================================================";
    cout << "\n   UBAH BARANG";
    cout << "\n==================================================";

    if (jumlah == 0) {
        cout << "\nBelum ada data barang.";
        cout << "\nTekan Enter untuk melanjutkan...";
        cin.get();
        return;
    }

    string kodeCari;
    cout << "\nMasukkan kode barang: ";
    cin >> kodeCari;
    cin.ignore();

    int posisiBarang = cariIndex(data, 0, jumlah, kodeCari);

    if (posisiBarang != -1) {
        cout << "\nNama Barang baru: ";
        getline(cin, data[posisiBarang].namaBarang);
        cout << "Kategori baru: ";
        getline(cin, data[posisiBarang].kategori);
        cout << "Jumlah baru: ";
        cin >> data[posisiBarang].jumlah;
        cout << "Harga baru: ";
        cin >> data[posisiBarang].harga;
        cin.ignore();
        cout << "Gedung baru: ";
        getline(cin, data[posisiBarang].alamat.gedung);
        cout << "Lantai baru: ";
        getline(cin, data[posisiBarang].alamat.lantai);
        cout << "Rak baru: ";
        getline(cin, data[posisiBarang].alamat.rak);

        data[posisiBarang].tersedia = (data[posisiBarang].jumlah > 0);

        cout << "\nData berhasil diubah.";
    } else {
        cout << "\nBarang tidak ditemukan.";
    }
    cout << "\nTekan Enter untuk melanjutkan...";
    cin.get();
}
```

F. Fitur Hapus Barang

Prosedur `hapusBarang` berfungsi menghapus data barang berdasarkan kode. Program pertama-tama mengecek apakah data kosong (`jumlah == 0`), lalu meminta input `kodeHapus` dan mencarinya menggunakan fungsi `cariIndex`. Jika barang ditemukan, program meminta konfirmasi pengguna (y/n); jika dikonfirmasi, data dihapus dengan menggeser elemen array setelah posisi tersebut ke kiri (`data[i] = data[i + 1]`), variabel `jumlah` dikurangi 1, dan pesan sukses ditampilkan. Jika barang tidak ditemukan atau pengguna membatalkan, program menampilkan pesan sesuai kondisi. Terakhir, program menunggu pengguna menekan Enter untuk melanjutkan.

Source Code:

```
void hapusBarang(ItemInventaris data[], int &jumlah) { // Prosedur
    cout << "\n==================================================";
    cout << "\n   HAPUS BARANG";
    cout << "\n==================================================";

    if (jumlah == 0) {
        cout << "\nBelum ada data barang.";
        cout << "\nTekan Enter untuk melanjutkan...";
        cin.get();
        return;
    }

    string kodeHapus;
    cout << "\nMasukkan kode barang: ";
    cin >> kodeHapus;
    cin.ignore();

    int posisiBarang = cariIndex(data, 0, jumlah, kodeHapus);

    if (posisiBarang != -1) {
        string konfirmasi;
        cout << "Yakin ingin menghapus? (y/n): ";
        getline(cin, konfirmasi);

        if (konfirmasi == "y" || konfirmasi == "Y") {
            for (int i = posisiBarang; i < jumlah - 1; i++) {
                data[i] = data[i + 1];
            }
            jumlah--;
            cout << "\nBarang berhasil dihapus!";
        } else {
            cout << "\nPenghapusan dibatalkan.";
        }
```

```
    } else {
        cout << "\nBarang tidak ditemukan.";
    }
    cout << "\nTekan Enter untuk melanjutkan...";
    cin.get();
}
```

4. Hasil Output

```
==================================================
   SISTEM MANAJEMEN INVENTARIS GUDANG ELEKTRONIK
==================================================
1. Login
2. Keluar Program

Pilihan: 1

--- Login (Percobaan 1/3) ---
Username: Pirlol
Password: 008
Username atau Password salah!
Tekan Enter untuk mencoba lagi...

--- Login (Percobaan 2/3) ---
Username: Acikiwir
Password: 008
Username atau Password salah!
Tekan Enter untuk mencoba lagi...

--- Login (Percobaan 3/3) ---
Username: Pirlo
Password: hengker tyz
Username atau Password salah!

Gagal login 3 kali. Program berhenti.
Tekan Enter untuk keluar...
```

Gambar 4.1 Output Login Gagal

```
--- Login (Percobaan 1/3) ---
Username: Pirlo
Password: 008

Login Berhasil!
Selamat datang.

Tekan Enter untuk melanjutkan...
```

Gambar 4.2 Output Login Berhasil

```
==================================================
   MENU UTAMA INVENTARIS GUDANG ELEKTRONIK
==================================================
1. Tambah Barang
2. Lihat Barang
3. Ubah Barang
4. Hapus Barang
5. Logout

Pilihan:
```

Gambar 4.3 Output Menu Utama

```
--- Tambah Barang ---
Kode Barang: EL004
Nama Barang: Kipas
Kategori: Alat Rumah Tangga
Jumlah: 12
Harga: 500000

--- Lokasi Penyimpanan ---
Gedung: Gudang C
Lantai: 1
Rak: 01

Barang berhasil ditambahkan!
Tekan Enter untuk melanjutkan...
```

Gambar 4.4 Output Tambah Barang

```
==================================================
   DAFTAR INVENTARIS GUDANG
==================================================
+-------+----------------+-------------------+--------+------------+---------------+----------+
|  Kode |   Nama Barang  |      Kategori     | Jumlah |    Harga   |     Lokasi    |  Status  |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL001 | Laptop Asus    | Komputer          | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL002 | Mouse Logitech | Aksesoris         | 50     | Rp 150000  | Gudang A-1-02 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL003 | Monitor LG     | Komputer          | 0      | Rp 2000000 | Gudang B-2-05 | Habis    |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL004 | Kipas          | Alat Rumah Tangga | 12     | Rp 500000  | Gudang C-1-01 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.5 Output Lihat Barang

```
Masukkan kode barang: EL003

Nama Barang baru: Kulkas
Kategori baru: Alat Rumah Tangga
Jumlah baru: 15
Harga baru: 1200000
Gedung baru: Gudang C
Lantai baru: 1
Rak baru: 02

Data berhasil diubah.
Tekan Enter untuk melanjutkan...
```

Gambar 4.6 Output Ubah Data Barang

```
==================================================
   DAFTAR INVENTARIS GUDANG
==================================================
+-------+----------------+-------------------+--------+------------+---------------+----------+
|  Kode |   Nama Barang  |      Kategori     | Jumlah |    Harga   |     Lokasi    |  Status  |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL001 | Laptop Asus    | Komputer          | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL002 | Mouse Logitech | Aksesoris         | 50     | Rp 150000  | Gudang A-1-02 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL003 | Kulkas         | Alat Rumah Tangga | 15     | Rp 1200000 | Gudang C-1-02 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+
| EL004 | Kipas          | Alat Rumah Tangga | 12     | Rp 500000  | Gudang C-1-01 | Tersedia |
+-------+----------------+-------------------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.7 Output Setelah Ubah Data Barang

```
Masukkan kode barang: EL002
Yakin ingin menghapus? (y/n): y

Barang berhasil dihapus!
Tekan Enter untuk melanjutkan...
```

Gambar 4.8 Output Hapus Barang

```
==================================================
   DAFTAR INVENTARIS GUDANG
==================================================
+-------+-------------+-------------------+--------+------------+---------------+----------+
|  Kode | Nama Barang |      Kategori     | Jumlah |    Harga   |     Lokasi    |  Status  |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL001 | Laptop Asus | Komputer          | 10     | Rp 5000000 | Gudang A-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL003 | Kulkas      | Alat Rumah Tangga | 15     | Rp 1200000 | Gudang C-1-02 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+
| EL004 | Kipas       | Alat Rumah Tangga | 12     | Rp 500000  | Gudang C-1-01 | Tersedia |
+-------+-------------+-------------------+--------+------------+---------------+----------+

Tekan Enter untuk melanjutkan...
```

Gambar 4.9 Output Setelah Hapus Barang

```
==================================================
   SISTEM MANAJEMEN INVENTARIS GUDANG ELEKTRONIK
==================================================
1. Login
2. Keluar Program

Pilihan: 2

Terima kasih telah menggunakan program ini.
```

Gambar 4.10 Output Keluar dari Program

5. Langkah-langkah GIT

5.1 GIT Init

Digunakan untuk membuat repository Git baru di folder lokal, sehingga folder tersebut siap melacak semua perubahan yang terjadi pada file di dalamnya.

```
PS C:\praktikum-apl\post-test> git init
Initialized empty Git repository in C:/praktikum-apl/post-test/.git/
```

Gambar 5.1 Git Init

5.2 GIT Add

Digunakan untuk menandai perubahan file tertentu agar masuk ke daftar antrean persiapan sebelum disimpan.

```
PS C:\praktikum-apl\post-test> git add .
```

Gambar 5.2 Git Add

5.3 GIT Commit

Digunakan untuk menyimpan semua perubahan yang ada di daftar antrean ke dalam riwayat proyek secara permanen beserta pesan penjelasannya.

```
PS C:\praktikum-apl> git commit -m "upload file pt 3"
[main 9ce5e49] upload file pt 3
 2 files changed, 322 insertions(+)
 create mode 100644 post-test/post-test-apl-3/2509106008-PirloSyabilaHafuza-PT-3.cpp
 create mode 100644 post-test/post-test-apl-3/2509106008-PirloSyabilaHafuza-PT-3.exe
```

Gambar 5.3 Git Commit

5.4 GIT Remote

Digunakan untuk menghubungkan repository lokal dengan alamat repository online di GitHub sebagai tujuan penyimpanan.

```
PS C:\praktikum-apl\post-test> git remote add origin https://github.com/limitless1fever/praktikum-apl.git
```

Gambar 5.4 Git Remote

5.5 GIT Push

Digunakan untuk mengirimkan seluruh riwayat perubahan yang telah disimpan dari komputer lokal ke repository online di GitHub.

```
PS C:\praktikum-apl> git push -u origin main
Enumerating objects: 12, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 2 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (8/8), 816.83 KiB | 1.92 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/limitless1fever/praktikum-apl
   68db0c8..5fe12f7  main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.5 Git Push